LAPORAN TUGAS SISTEM OPERASI (PRAKTIKUM)

TUGAS P4: USER MANAGEMENT AND PERMISSION

STUDI KASUS PT. DIGITAL INTELIJENSIA CIWARUGA

Laporan Ini Disusun Untuk Memenuhi Tugas Kuliah

Mata Kuliah : Sistem Operasi

Disusun Oleh :

Nama : Ghaisan Khoirul Badruzaman

NIM : 251524048

Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA

JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA

POLITEKNIK NEGERI BANDUNG

2026

DAFTAR ISI

BAB I PENDAHULUAN **3**

1.1 Studi Kasus 3

1.2 Lingkungan Pengerjaan 3

1.3 Konsep yang Dipakai 3

BAB II MANAJEMEN USER DAN GROUP **5**

2.1 Soal 1.1: Perintah membuat user dan group 5

2.2 Soal 1.2: Verifikasi keanggotaan grup 6

BAB III STRUKTUR FOLDER DAN PERMISSION DASAR **7**

3.1 Soal 2.1: Mengatur kepemilikan folder 7

3.2 Soal 2.2: Pengaturan permission 8

BAB IV FILE AKSES KHUSUS **11**

4.1 Soal 3.1: Mengatur hak akses report_finance.txt dengan ACL 11

4.2 Soal 3.2: Verifikasi aturan akses 12

BAB V INHERITANCE DAN SETGID **14**

5.1 Soal 4.1: Permission khusus untuk folder developer 14

5.2 Soal 4.2: Menguji pewarisan grup 15

BAB VI SKENARIO KHUSUS **16**

6.1 Soal 5.1: Akses penuh andi ke projectX dengan ACL 16

6.2 Soal 5.2: Verifikasi akses penuh andi 17

BAB VII PENUTUP **19**

7.1 Ringkasan Konfigurasi Akhir 19

7.2 Catatan 19

7.3 Kesimpulan 19

BAB I PENDAHULUAN

1.1 Studi Kasus

PT. Digital Intelijensia Ciwaruga memakai satu server Linux untuk dokumen internal tiga divisi: Marketing, Developer, dan Finance. Tiap divisi punya folder kerja sendiri, ada folder bersama yang cuma boleh dibaca, ada satu file keuangan yang dibagi ke Marketing dengan hak baca saja, folder Developer harus mewariskan grupnya ke file baru, dan ada user luar bernama andi yang butuh akses penuh ke satu folder proyek. Laporan ini menjawab kelima bagian soal. Tiap jawaban berisi command line, penjelasan, dan screenshot terminal asli.

1.2 Lingkungan Pengerjaan

Supaya konfigurasi user dan grup tidak tercampur dengan mesin Linux yang dipakai di tugas sebelumnya, dibuat mesin Ubuntu baru di OrbStack bernama `ciwaruga`, sesuai nama server di studi kasus. Semua perintah dijalankan oleh akun admin `ghaisan` yang punya hak sudo, dan zona waktunya disetel ke WIB.

Tabel 1.1: Versi perangkat lunak di mesin ciwaruga

Komponen	Versi
Sistem operasi	Ubuntu 26.04.1 LTS (OrbStack)
Kernel	Linux 7.0.14-orbstack aarch64
Shell	GNU bash 5.3.9
`useradd`, `groupadd`	passwd (shadow-utils) 4.17.4
`ls`, `chmod`, `chown`, `id`	uutils coreutils 0.10.0
`sudo`	sudo-rs 0.2.13
`setfacl`, `getfacl`	acl 2.3.2
Filesystem	btrfs (mendukung POSIX ACL)

Ubuntu 26.04 memakai coreutils versi Rust (uutils) dan sudo-rs sebagai bawaan, bukan GNU coreutils dan sudo klasik. Untuk perintah yang dipakai di tugas ini keluarannya sama dengan versi GNU, termasuk tanda `+` pada `ls -l` untuk file yang punya ACL. Paket `acl` dipasang dulu dengan `apt install acl` karena image bawaan belum menyertakan `setfacl` dan `getfacl`.

1.3 Konsep yang Dipakai

Setiap file dan folder punya pemilik, grup, dan tiga kelompok hak akses: owner, group, dan other. Arti hak akses berbeda antara file dan folder, dan perbedaan ini yang paling sering menentukan berhasil tidaknya aturan akses di tugas ini.

Tabel 1.2: Arti r, w, x pada file dan folder

Hak	Pada file	Pada folder
`r` (4)	membaca isi file	melihat daftar isi folder (`ls`)
`w` (2)	mengubah isi file	membuat, menghapus, dan mengganti nama isi folder (butuh `x` juga)
`x` (1)	menjalankan file sebagai program	masuk dan melewati folder untuk membuka isinya

Permission dasar hanya punya satu slot grup. Kalau lebih dari satu grup butuh hak berbeda, dipakai ACL (Access Control List) yang menambah entri untuk user atau grup tertentu. ACL punya entri `mask` yang membatasi hak maksimal semua entri grup dan user tambahan, serta default ACL yang otomatis diwariskan ke file dan folder baru di dalam sebuah folder. Selain itu ada bit khusus setgid yang membuat file baru di sebuah folder mengikuti grup folder itu.

Pengujian akses dilakukan dengan `sudo -u <user> <perintah>`, yaitu menjalankan perintah sebagai user tertentu lengkap dengan grup-grupnya. Dengan cara ini tiap aturan bisa dibuktikan berhasil dan gagal tanpa perlu login bergantian ke akun masing-masing.

BAB II MANAJEMEN USER DAN GROUP

2.1 Soal 1.1: Perintah membuat user dan group

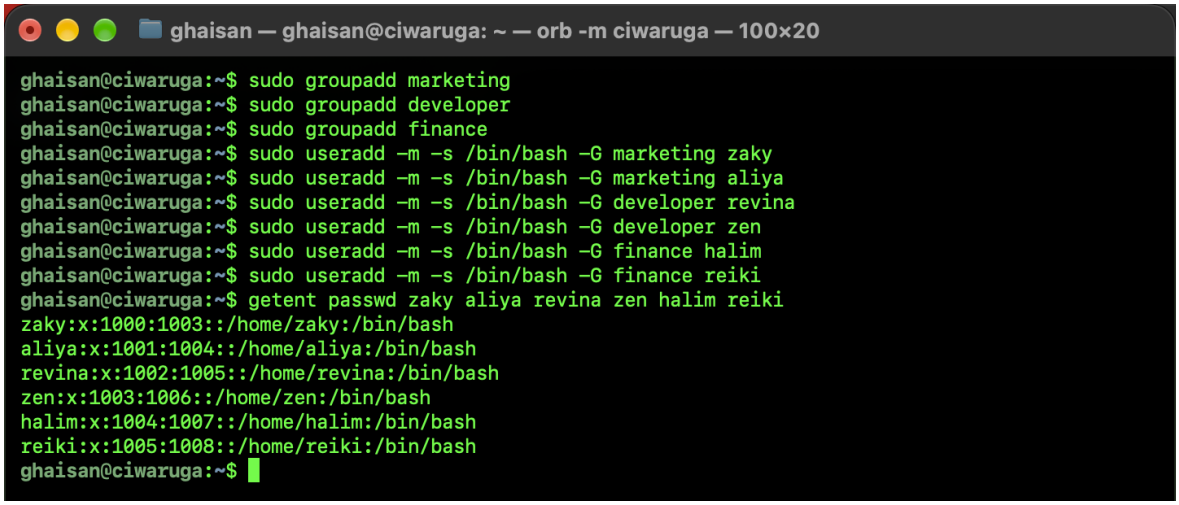
Pertanyaan. Perintah apa yang digunakan untuk membuat user dan group tersebut?

Command line.

```
sudo groupadd marketing
sudo groupadd developer
sudo groupadd finance
sudo useradd -m -s /bin/bash -G marketing zaky
sudo useradd -m -s /bin/bash -G marketing aliya
sudo useradd -m -s /bin/bash -G developer revina
sudo useradd -m -s /bin/bash -G developer zen
sudo useradd -m -s /bin/bash -G finance halim
sudo useradd -m -s /bin/bash -G finance reiki
getent passwd zaky aliya revina zen halim reiki
```

Penjelasan. `groupadd` membuat grup baru dan mencatatnya di `/etc/group`. `useradd` membuat user dengan opsi `-m` untuk membuat folder home dari `/etc/skel`, `-s /bin/bash` untuk shell login, dan `-G` untuk memasukkan user ke grup divisinya sebagai grup tambahan (supplementary group). Terakhir `getent passwd` dipakai untuk melihat keenam user sudah tercatat beserta home dan shell-nya.

Di Ubuntu, `USERGROUPS_ENAB` bernilai `yes`, jadi setiap user otomatis mendapat grup pribadi dengan nama sama sebagai grup utama (misalnya zaky dengan GID 1003). Grup divisi sengaja dijadikan grup tambahan supaya file pribadi user tetap milik grup pribadinya. Kalau user sudah ada, cara menambahkannya ke grup adalah `usermod -aG marketing zaky`; opsi `-a` wajib, karena tanpa itu grup tambahan lain milik user akan terhapus. Akun yang dibuat `useradd` belum punya password dan terkunci sampai diberi password dengan `passwd`.



Gambar 2.1: membuat tiga grup dan enam user

2.2 Soal 1.2: Verifikasi keanggotaan grup

Pertanyaan. Bagaimana cara verifikasi bahwa user sudah benar tergabung dalam grup?

Command line.

```
id zaky
id aliya
id revina
id zen
id halim
id reiki
getent group marketing developer finance
```

Penjelasan. `id <user>` menampilkan UID, grup utama (gid), dan semua grup yang diikuti user. Hasilnya menunjukkan zaky dan aliya masuk `marketing`, revina dan zen masuk `developer`, serta halim dan reiki masuk `finance`, masing-masing di samping grup pribadinya. Dari arah sebaliknya, `getent group` menampilkan daftar anggota tiap grup langsung dari database grup. Perintah `groups <user>` juga bisa dipakai, hasilnya sama dengan kolom groups pada `id`.



Gambar 2.2: verifikasi keanggotaan grup dengan id dan getent

BAB III STRUKTUR FOLDER DAN PERMISSION DASAR

3.1 Soal 2.1: Mengatur kepemilikan folder

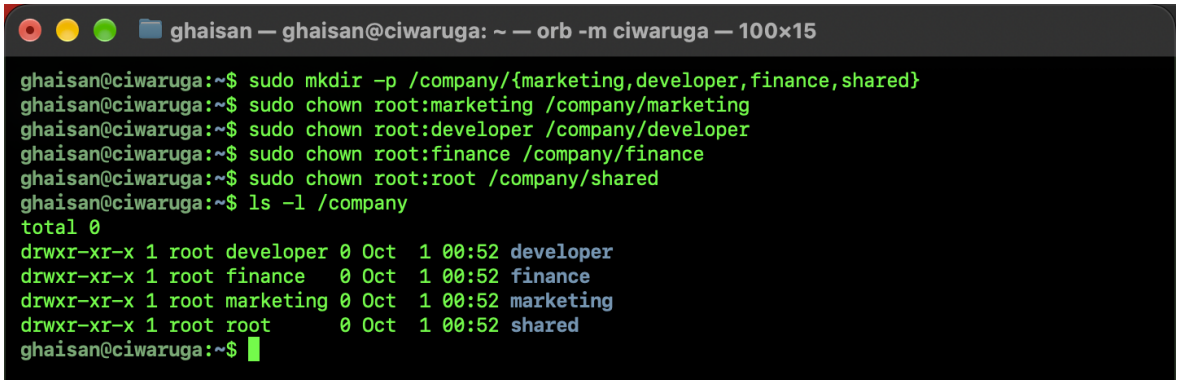
Pertanyaan. Perintah apa yang Anda gunakan untuk mengatur kepemilikan folder tersebut?

Command line.

```
sudo mkdir -p /company/{marketing,developer,finance,shared}
sudo chown root:marketing /company/marketing
sudo chown root:developer /company/developer
sudo chown root:finance /company/finance
sudo chown root:root /company/shared
ls -l /company
```

Penjelasan. `mkdir -p` dengan brace expansion membuat keempat folder sekaligus. `chown root:<grup>` mengatur pemilik dan grup folder dalam satu perintah. Pemiliknya root supaya anggota divisi tidak bisa mengubah permission foldernya sendiri, karena hanya pemilik dan root yang boleh menjalankan `chmod`. Grupnya diisi grup divisi, sehingga hak akses bagian group nanti berlaku untuk seluruh anggota divisi itu. Folder `shared` tetap milik `root:root` karena harus dibaca tiga grup sekaligus, dan itu diatur lewat ACL di soal 2.2.

Terlihat di `ls -l` bahwa permission keempat folder masih `drwxr-xr-x` bawaan umask, artinya semua orang masih bisa masuk. Permission ini diperketat di soal berikutnya.



Gambar 3.1: membuat folder dan mengatur pemilik serta grupnya

3.2 Soal 2.2: Pengaturan permission

Pertanyaan. Bagaimana pengaturan permission agar sesuai dengan aturan di atas?

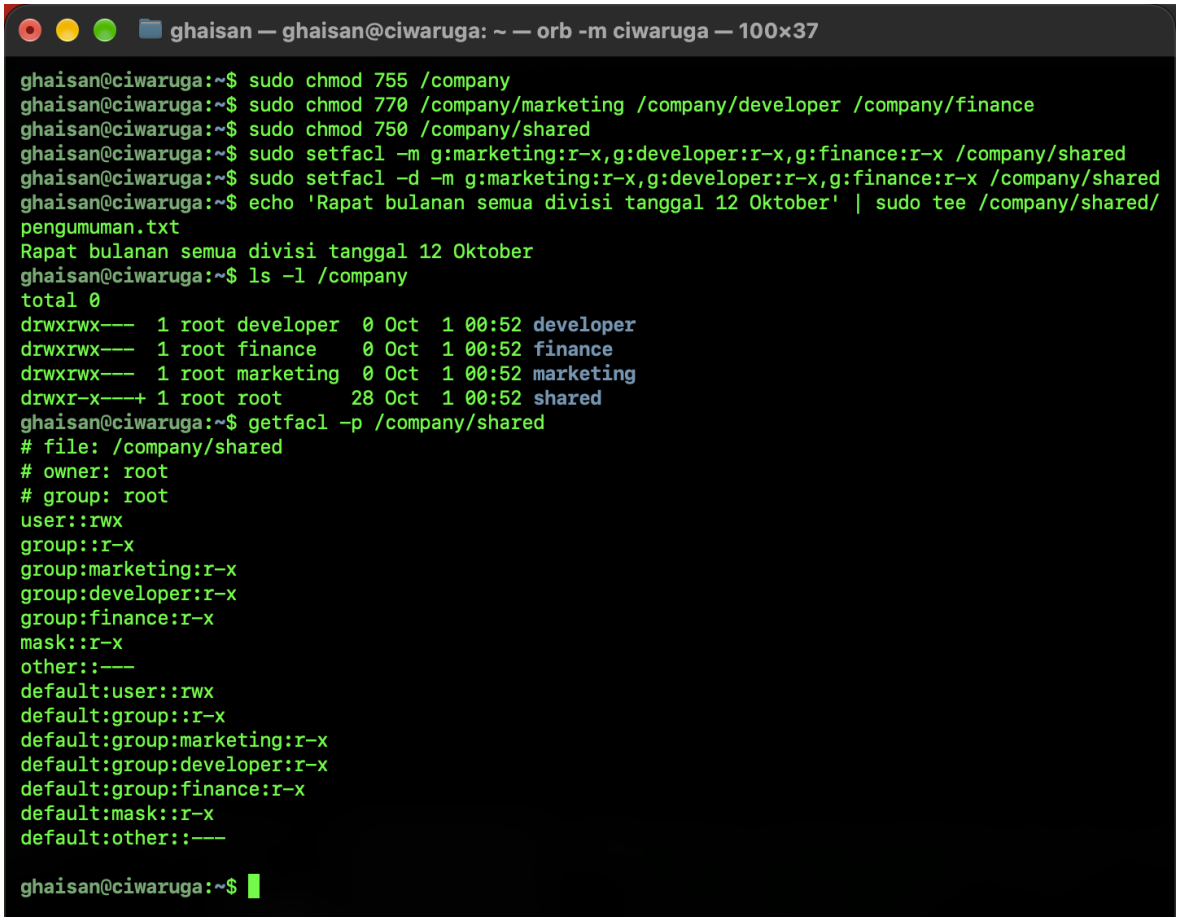
Command line (konfigurasi).

```
sudo chmod 755 /company
sudo chmod 770 /company/marketing /company/developer /company/finance
sudo chmod 750 /company/shared
sudo setfacl -m g:marketing:r-x,g:developer:r-x,g:finance:r-x /company/shared
sudo setfacl -d -m g:marketing:r-x,g:developer:r-x,g:finance:r-x /company/shared
echo 'Rapat bulanan semua divisi tanggal 12 Oktober' | sudo tee /company/shared/peng
umuman.txt
ls -l /company
getfacl -p /company/shared
```

Penjelasan. `/company` diberi `755` supaya semua user bisa melewatinya menuju folder divisi; tanpa hak `x` di sini tidak ada satu pun subfolder yang bisa dicapai. Folder divisi diberi `770`: pemilik root dan grup divisi mendapat `rwx`, sedangkan other tidak mendapat apa-apa, jadi hanya anggota grupnya yang bisa masuk.

Folder `shared` harus bisa dibaca tiga grup, padahal permission dasar hanya punya satu slot grup. Karena itu foldernya dibuat `750` milik `root:root`, lalu `setfacl -m` menambahkan entri `r-x` untuk grup marketing, developer, dan finance. `r` untuk melihat daftar isi, `x` untuk masuk dan membuka file, dan tanpa `w` anggota grup tidak bisa membuat, mengubah, atau menghapus isi folder. Opsi `-d` membuat default ACL yang sama, sehingga file baru di dalam `shared` (contohnya `pengumuman.txt` yang dibuat admin) otomatis bisa dibaca ketiga grup. Tanda `+` pada `drwxr-x---+` menunjukkan folder itu punya ACL.

Cara ini lebih ketat dibanding `chmod 755` biasa: dengan 755 semua user di server bisa membaca `shared`, termasuk user yang bukan anggota divisi mana pun, sedangkan dengan ACL hanya tiga grup divisi yang bisa.

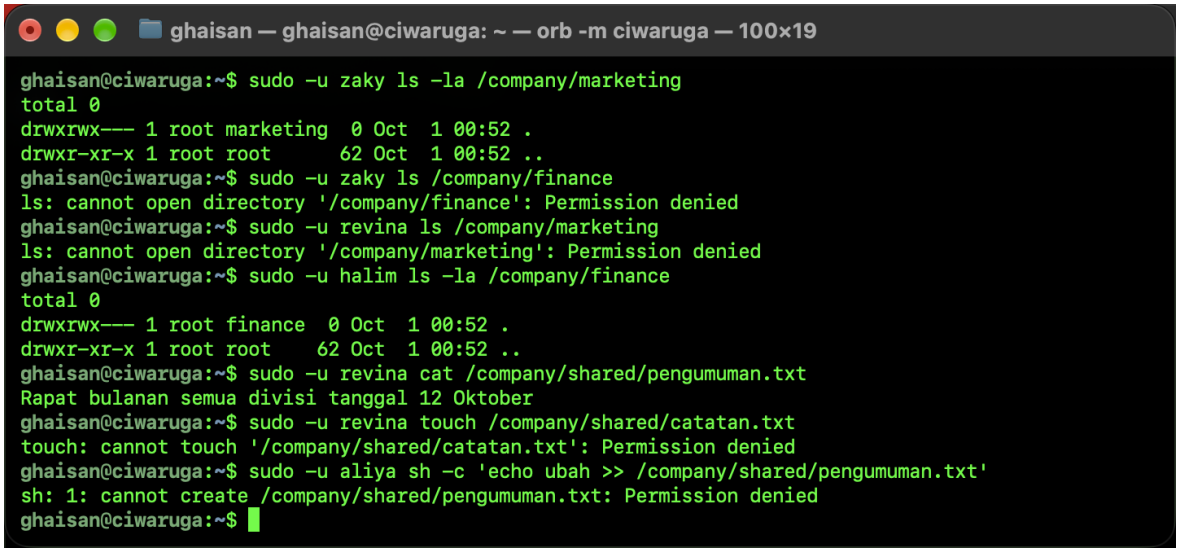


Gambar 3.2: mengatur permission folder divisi dan ACL folder shared

Command line (pengujian).

```
sudo -u zaky ls -la /company/marketing
sudo -u zaky ls /company/finance
sudo -u revina ls /company/marketing
sudo -u halim ls -la /company/finance
sudo -u revina cat /company/shared/pengumuman.txt
sudo -u revina touch /company/shared/catatan.txt
sudo -u aliya sh -c 'echo ubah >> /company/shared/pengumuman.txt'
```

Pengujian. zaky (marketing) bisa masuk ke folder marketing tetapi ditolak di folder finance, revina (developer) ditolak di folder marketing, dan halim (finance) bisa masuk ke folder finance. Di folder `shared`, revina bisa membaca `pengumuman.txt` tetapi gagal membuat file baru, dan aliya gagal mengubah isi pengumuman. Semua hasil ini sesuai aturan: folder divisi tertutup untuk divisi lain, folder bersama hanya bisa dibaca.



Gambar 3.3: menguji akses folder divisi dan folder shared

BAB IV FILE AKSES KHUSUS

4.1 Soal 3.1: Mengatur hak akses report_finance.txt dengan ACL

Pertanyaan. Bagaimana cara mengatur hak akses tersebut menggunakan ACL?

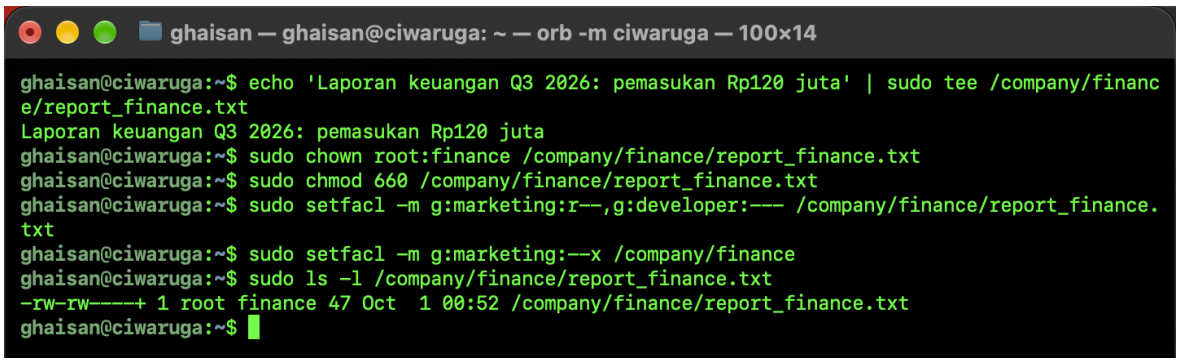
Command line.

```
echo 'Laporan keuangan Q3 2026: pemasukan Rp120 juta' | sudo tee /company/finance/re
port_finance.txt
sudo chown root:finance /company/finance/report_finance.txt
sudo chmod 660 /company/finance/report_finance.txt
sudo setfacl -m g:marketing:r--,g:developer:--- /company/finance/report_finance.txt
sudo setfacl -m g:marketing:--x /company/finance
sudo ls -l /company/finance/report_finance.txt
```

Penjelasan. File dibuat lewat `sudo tee`, lalu dijadikan milik `root:finance` dengan mode `660`: grup finance bisa membaca dan menulis, other tidak punya akses. ACL lalu menambah dua entri: `g:marketing:r--` supaya marketing bisa membaca tanpa menulis, dan `g:developer:---` sebagai larangan eksplisit untuk developer.

Entri `---` untuk developer tidak sekadar pelengkap. Saat memeriksa akses, Linux mencocokkan entri grup lebih dulu daripada other, jadi anggota developer tetap ditolak walaupun suatu saat hak other dibuka.

Bagian yang paling mudah terlewat adalah folder `/company/finance` yang bermode `770`. ACL pada file saja tidak cukup, karena marketing tidak bisa melewati folder itu untuk sampai ke file. Karena itu folder diberi `g:marketing:--x`: marketing boleh melewati folder dan membuka file yang namanya sudah diketahui, tetapi tetap tidak bisa melihat daftar isi folder karena tidak punya `r`.



Gambar 4.1: mengatur ACL report_finance.txt dan folder finance

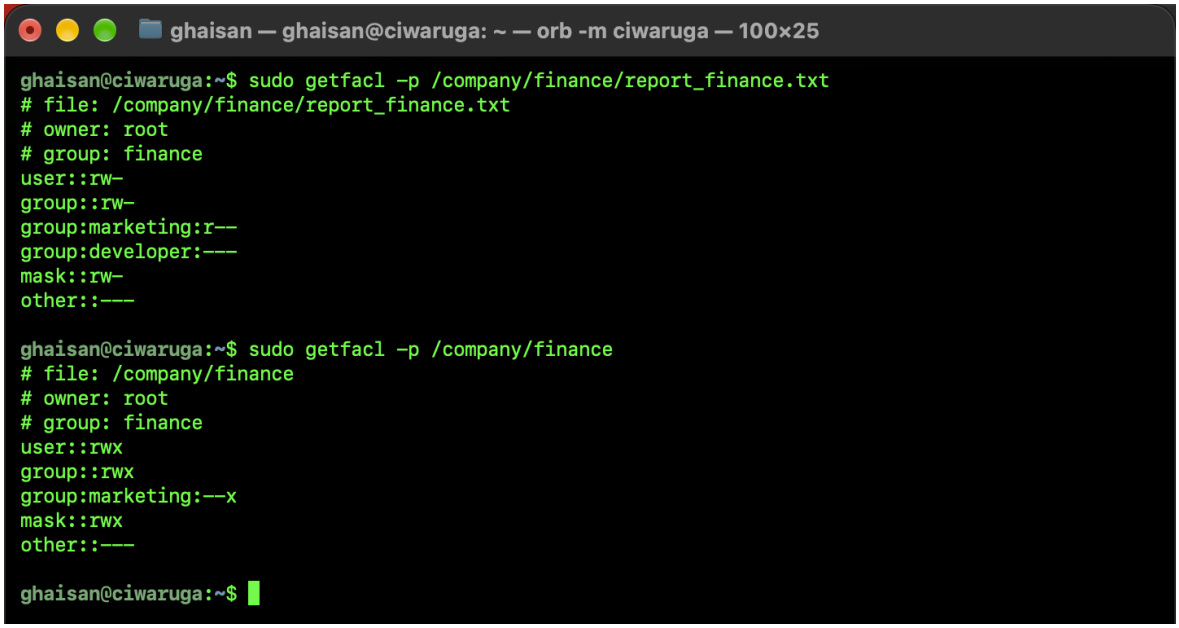
4.2 Soal 3.2: Verifikasi aturan akses

Pertanyaan. Perintah apa untuk memverifikasi bahwa aturan sudah benar?

Command line (getfacl).

```
sudo getfacl -p /company/finance/report_finance.txt
sudo getfacl -p /company/finance
```

Penjelasan. `getfacl` menampilkan seluruh entri ACL. Pada file terlihat `group::rw-` untuk finance, `group:marketing:r--`, `group:developer:---`, dan `mask::rw-`. Mask adalah batas atas hak semua entri grup dan user tambahan; di sini `rw-` sehingga hak marketing tetap `r--`. Pada folder terlihat entri `group:marketing:--x`. Tanda `+` pada `ls -l` di soal 3.1 juga menunjukkan file itu punya ACL.

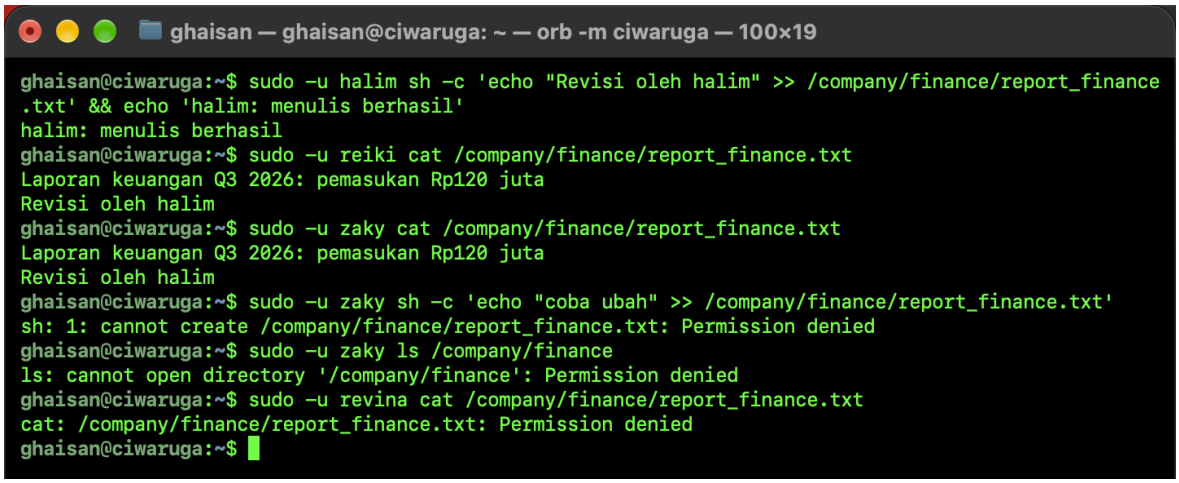


Gambar 4.2: melihat ACL file dan folder dengan getfacl

Command line (pengujian).

```
sudo -u halim sh -c 'echo "Revisi oleh halim" >> /company/finance/report_finance.txt
' && echo 'halim: menulis berhasil'
sudo -u reiki cat /company/finance/report_finance.txt
sudo -u zaky cat /company/finance/report_finance.txt
sudo -u zaky sh -c 'echo "coba ubah" >> /company/finance/report_finance.txt'
sudo -u zaky ls /company/finance
sudo -u revina cat /company/finance/report_finance.txt
```

Pengujian. halim berhasil menambah baris ke file, dan reiki (sesama finance) bisa membaca hasilnya. zaky (marketing) bisa membaca isi file tetapi gagal menulis, dan tetap tidak bisa melihat daftar isi folder finance. revina (developer) langsung ditolak saat membaca. Ketiga aturan di soal terbukti berjalan.



Gambar 4.3: menguji akses finance, marketing, dan developer ke report_finance.txt

BAB V INHERITANCE DAN SETGID

5.1 Soal 4.1: Permission khusus untuk folder developer

Pertanyaan. Permission khusus apa yang harus diatur pada folder tersebut?

Command line.

```
sudo -u revina touch /company/developer/sebelum.txt
sudo ls -l /company/developer
sudo chmod g+s /company/developer
sudo setfacl -d -m g::rwx,o::--- /company/developer
ls -ld /company/developer
sudo getfacl -p /company/developer
```

Penjelasan. Yang dipakai adalah bit **setgid** pada folder, dengan `chmod g+s` (atau dalam angka `2770`). Pada folder, setgid membuat file dan subfolder baru mengikuti grup folder induknya, bukan grup utama user yang membuatnya. Subfolder baru juga ikut mewarisi bit setgid. Setelah dipasang, huruf `x` pada bagian group berubah menjadi `s` (`drwxrws---`) dan `getfacl` menampilkan `flags: -s-`.

Untuk pembanding, sebelum setgid dipasang revina membuat `sebelum.txt`, dan file itu jatuh ke grup pribadi `revina`. Sebagai tambahan dipasang default ACL `g::rwx,o::---`. Setgid hanya mengatur grup, bukan hak akses, dan umask 022 bawaan akan membuat file baru bermode `644` sehingga rekan satu grup tidak bisa mengubahnya. Dengan default ACL, file baru di folder developer otomatis bisa ditulis anggota grup.



Gambar 5.1: memasang setgid dan default ACL di folder developer

5.2 Soal 4.2: Menguji pewarisan grup

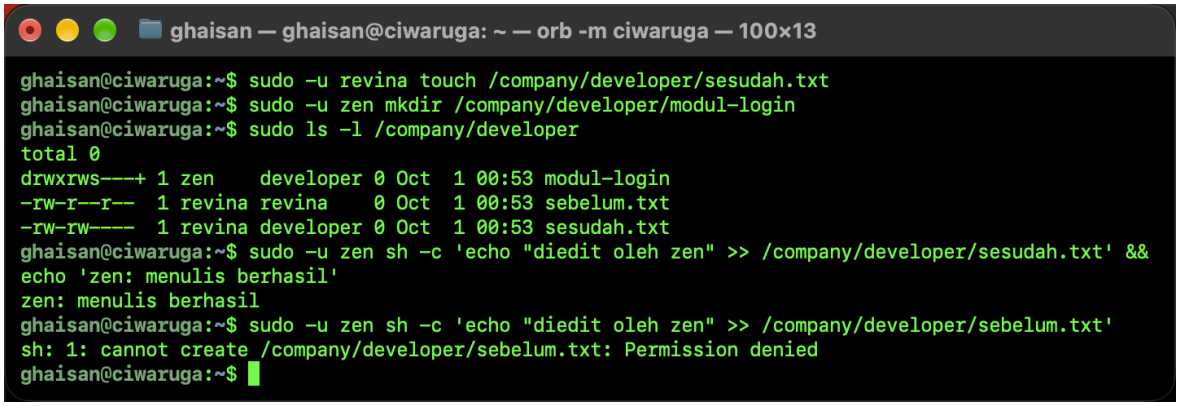
Pertanyaan. Bagaimana cara menguji apakah aturan sudah berjalan dengan benar?

Command line.

```
sudo -u revina touch /company/developer/sesudah.txt
sudo -u zen mkdir /company/developer/modul-login
sudo ls -l /company/developer
sudo -u zen sh -c 'echo "diedit oleh zen" >> /company/developer/sesudah.txt' && echo
 'zen: menulis berhasil'
sudo -u zen sh -c 'echo "diedit oleh zen" >> /company/developer/sebelum.txt'
```

Penjelasan. Pengujiannya dengan membuat file dan folder baru sebagai anggota developer, lalu melihat grupnya dengan `ls -l`. `sesudah.txt` buatan revina jatuh ke grup `developer`, padahal `sebelum.txt` yang dibuat sebelum setgid ada di grup `revina`. Folder `modul-login` buatan zen juga bergrup `developer` dan ikut bertanda `s`, jadi pewarisan berlanjut ke subfolder.

Pengaruhnya langsung terasa saat bekerja sama: zen berhasil menulis ke `sesudah.txt` karena file itu milik grup developer dan bermode `rw-rw----`, tetapi gagal menulis ke `sebelum.txt` yang masih milik grup revina.



Gambar 5.2: menguji pewarisan grup pada file dan subfolder baru

BAB VI SKENARIO KHUSUS

6.1 Soal 5.1: Akses penuh andi ke projectX dengan ACL

Pertanyaan. Bagaimana cara mengatur permission ini dengan ACL?

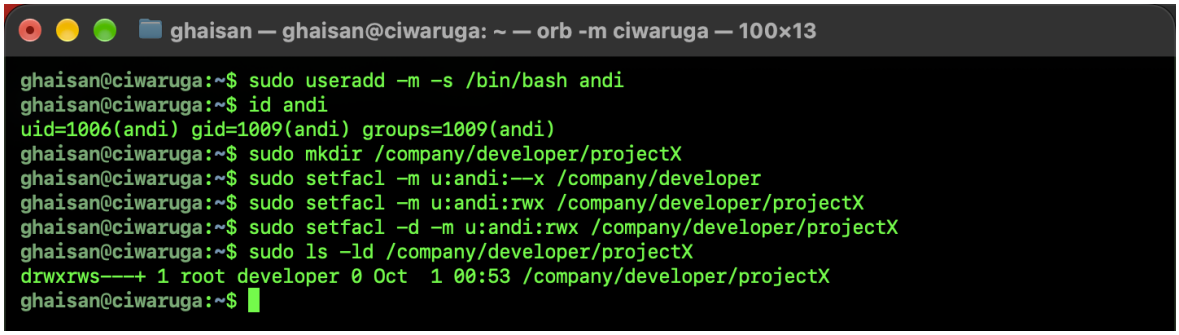
Command line.

```
sudo useradd -m -s /bin/bash andi
id andi
sudo mkdir /company/developer/projectX
sudo setfacl -m u:andi:--x /company/developer
sudo setfacl -m u:andi:rwx /company/developer/projectX
sudo setfacl -d -m u:andi:rwx /company/developer/projectX
sudo ls -ld /company/developer/projectX
```

Penjelasan. User andi dibuat tanpa grup divisi, jadi `id andi` hanya menunjukkan grup pribadinya. Folder `projectX` dibuat di dalam `/company/developer` yang sudah ber-setgid, sehingga otomatis bergrup `developer`. Aksesnya diberikan lewat tiga entri ACL:

1. `u:andi:--x` pada `/company/developer`, supaya andi bisa melewati folder developer menuju projectX, tetapi tidak bisa melihat daftar isinya maupun membuka file developer lain.
2. `u:andi:rwx` pada `projectX`, yaitu akses penuh baca, tulis, dan eksekusi.
3. Default ACL `u:andi:rwx` pada `projectX`, supaya file dan folder yang dibuat nanti di dalam projectX, oleh siapa pun, tetap bisa diakses andi.

Tidak ada perintah `chown` atau `chgrp` yang dijalankan, jadi pemilik dan grup utama projectX tetap `root:developer`, sesuai syarat soal.



Gambar 6.1: membuat user andi dan memberi ACL pada projectX

6.2 Soal 5.2: Verifikasi akses penuh andi

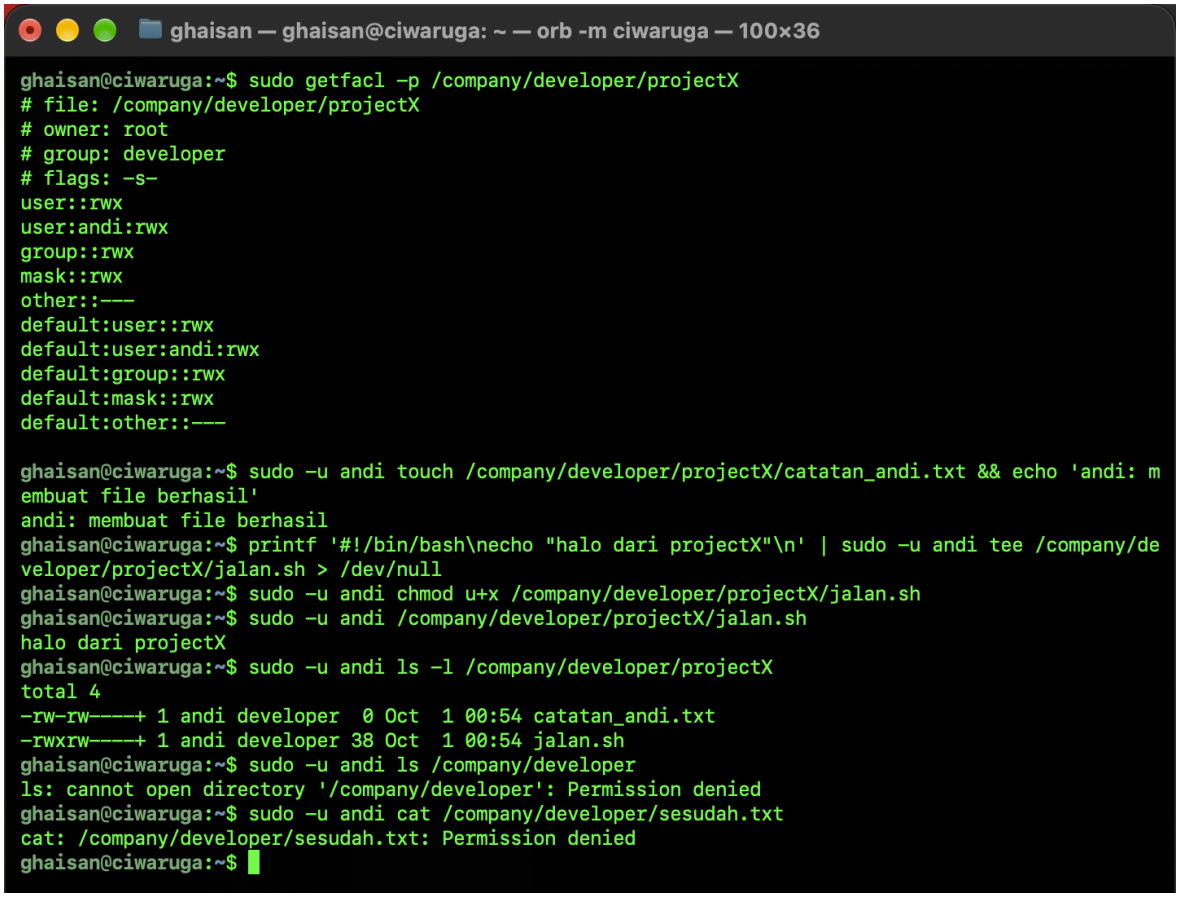
Pertanyaan. Perintah apa yang digunakan untuk memverifikasi bahwa andi memiliki akses penuh?

Command line.

```
sudo getfacl -p /company/developer/projectX
sudo -u andi touch /company/developer/projectX/catatan_andi.txt && echo 'andi: membu
at file berhasil'
printf '#!/bin/bash\necho "halo dari projectX"\n' | sudo -u andi tee /company/develo
per/projectX/jalan.sh > /dev/null
sudo -u andi chmod u+x /company/developer/projectX/jalan.sh
sudo -u andi /company/developer/projectX/jalan.sh
sudo -u andi ls -l /company/developer/projectX
sudo -u andi ls /company/developer
sudo -u andi cat /company/developer/sesudah.txt
```

Penjelasan. `getfacl` menunjukkan entri `user:andi:rwx` beserta default-nya, dengan `mask::rwx` sehingga hak andi tidak terpotong. Grup folder tetap `developer`. Ketiga hak lalu diuji satu per satu sebagai andi: tulis dengan membuat `catatan_andi.txt`, eksekusi dengan menjalankan skrip `jalan.sh` yang mencetak "halo dari projectX", dan baca dengan `ls -l` isi projectX.

File buatan andi juga jatuh ke grup `developer` karena projectX mewarisi setgid, jadi tim developer tetap bisa ikut mengerjakannya. Dua perintah terakhir membuktikan akses andi hanya sebatas projectX: andi tidak bisa melihat daftar isi `/company/developer` dan tidak bisa membaca `sesudah.txt` milik developer.



Gambar 6.2: menguji akses baca, tulis, dan eksekusi andi di projectX

BAB VII PENUTUP

7.1 Ringkasan Konfigurasi Akhir

Tabel 7.1: Pemilik, mode, dan ACL tiap path

Path	Pemilik:grup	Mode	ACL tambahan
`/company`	root:root	755	-
`/company/marketing`	root:marketing	770	-
`/company/finance`	root:finance	770	`g:marketing:--x`
`.../report_finance.txt`	root:finance	660	`g:marketing:r--`, `g:developer:---`
`/company/developer`	root:developer	2770	`u:andi:--x`, default `g::rwx`
`.../developer/projectX`	root:developer	2770	`u:andi:rwx` + default
`/company/shared`	root:root	750	`r-x` untuk tiga grup + default

7.2 Catatan

1. Hak `x` pada folder adalah gerbang. Marketing baru bisa membaca `report_finance.txt` dan andi baru bisa masuk projectX setelah folder induknya diberi `--x`. Tanpa itu, ACL pada file atau subfolder tidak ada gunanya.
2. Pada file yang punya ACL, bagian group di `ls -l` menampilkan mask, bukan hak grup pemilik. Menjalankan `chmod` pada file seperti itu mengubah mask dan bisa ikut memotong hak semua entri ACL.
3. Setgid hanya mewariskan grup. Supaya file baru juga bisa ditulis anggota grup, umask perlu diatur atau dipasang default ACL seperti di soal 4.1.
4. Entri ACL `---` untuk sebuah grup berfungsi sebagai larangan eksplisit, karena entri grup diperiksa lebih dulu daripada other.
5. `sudo -u` sangat membantu untuk menguji aturan dari sudut pandang tiap user tanpa perlu membuat password dan login bergantian.

7.3 Kesimpulan

Permission dasar (owner, group, other) cukup untuk aturan satu divisi satu folder, tetapi langsung kewalahan begitu satu objek harus diberi hak berbeda untuk beberapa grup atau untuk user dari luar grup. Di situ ACL mengisi kekurangannya tanpa perlu mengubah pemilik dan grup utama. Setgid melengkapi keduanya untuk kerja kelompok, karena file baru otomatis masuk ke grup yang benar. Hal yang paling penting dari praktikum ini adalah selalu menguji aturan dari sisi user yang seharusnya berhasil dan yang seharusnya ditolak, karena kesalahan seperti lupa hak `x` pada folder induk baru kelihatan saat diuji.